

Ph.D. Proposal

**Toward Multi-SLO-Aware Scheduling for Heterogeneous AI
Accelerators on Edge Devices**

June 23, 2026

Contents

1	Introduction	1
2	Related Work	3
2.1	AI on Resource-Constrained Edge Devices	3
2.2	Edge AI Schedulers	4
3	Proposed Research	5
3.1	Thrust I: Multi-SLO heterogeneous scheduling	6
3.1.1	Proposed Approach of Thrust 1	6
3.1.2	Evaluation Plan	7
3.2	Thrust II: Model-aware dynamic inference adaptation	9
3.2.1	Proposed Approach of Thrust 2	9
3.2.2	Evaluation Plan	11
3.3	Thrust III: Self-Calibrating Multi-SLO Adaptation	12
3.3.1	Proposed Features for Thrust 3	12
3.3.2	Proposed Approach for Thrust 3	13
3.3.3	Evaluation Plan	14
4	Preliminary Work	15
4.1	Thrust I: Multi-SLO heterogeneous scheduling	15
5	Timeline, Summary, and Research Contributions	17
5.1	Timeline	17
5.2	Summary and Research Contributions	17

Abstract

The rapid proliferation of edge artificial intelligence (AI) is redefining the traditional boundaries between computation, communication, and learning. While recent schedulers such as Convergo have demonstrated that multi-SLO coordination across heterogeneous edge accelerators can significantly enhance system efficiency, current approaches still treat deployed AI models as immutable, “black-box” workloads. This implicit separation between system optimization and model design fundamentally limits the achievable trade-offs among latency, accuracy, and energy consumption.

This research aims to explore a unified, cross-layer approach to edge-AI efficiency that progressively integrates scheduling intelligence, model-aware adaptation, and deployment-feedback-driven self-calibration. The first stage investigates system-level coordination through Convergo, an intelligent multi-objective scheduler that simultaneously manages heterogeneous accelerators, multi-tenancy, and dynamic SLO satisfaction. Building on these foundations, the second stage develops a model-aware dynamic inference framework featuring skip-aware neural architectures, elasticity characterization, and lightweight performance predictors capable of navigating large configuration spaces in real time. The final stage advances toward self-calibrating edge AI systems that continuously learn from deployment feedback, detect adaptation degradation caused by evolving operating conditions, and autonomously refine adaptation policies to sustain long-term multi-SLO satisfaction.

Together, these contributions aim to transform edge-AI inference from a collection of independently optimized components into an integrated, self-adaptive ecosystem, one in which system scheduling, model execution, and architectural design co-evolve to deliver sustainable, efficient, and resilient intelligence at the network edge.

Research Contributions.

This research develops a comprehensive framework that advances edge AI from intelligent scheduling to model-aware adaptation and ultimately to self-calibrating deployment intelligence. The first contribution establishes a multi-SLO scheduling framework capable of coordinating heterogeneous accelerators and simultaneously satisfying accuracy, throughput, and deadline requirements. The second contribution introduces model-aware dynamic inference adaptation through skip-aware neural architectures, elasticity characterization, and lightweight prediction models that efficiently navigate large configuration spaces while maintaining performance guarantees. The final contribution develops a deployment-feedback-driven self-calibration framework that continuously evaluates adaptation effectiveness, detects deployment drift, and autonomously refines adaptation behavior using historical deployment experience. Together, these contributions transform edge AI inference from static resource orchestration into a self-improving ecosystem capable of sustaining efficient and reliable multi-SLO operation under long-term deployment and evolving environmental conditions.

1 Introduction

Edge AI and edge inference systems are gaining significant attention due to the increasing deployment of AI services and the rapid development of pre-trained models, devices, and AI accelerators [1] [2] [3] [4] [5] [6]. The primary benefit of edge AI is its ability to offer low-latency inference services. By placing small yet capable resources close to end-users and data sources, edge computing facilitates AI computations directly on the device [2] [7] [8]. This approach significantly reduces reliance on cloud resources in remote data centers, thereby minimizing data transfer overheads.

However, edge AI has an intrinsic challenge: *it must operate on resource-constrained devices while meeting specific (often multiple) service-level objectives (SLOs)*, such as accuracy targets, inference deadline goals, or throughput requirements. To address this challenge and enhance edge AI’s performance, various approaches have been developed [9] [10] [11] [12] [13]. One approach is focused on enhancing AI inference on edge accelerators using AI multi-tenancy [2] [8] [7], which involves running multiple deep learning (DL) models concurrently. Another commonly used approach relies on model compression, which reduces the size of DL models, making them suitable for execution on resource-constrained edge devices [14] [15] [16] [17] [18].

Nevertheless, these methods often address only a subset of the challenges – optimizing either a single SLO or a single accelerator type, without providing an integrated solution that can handle multiple SLOs across heterogeneous accelerators. As a result, they remain insufficient to tackle the core challenges of edge AI, particularly when multiple SLOs (*e.g.*, accuracy, deadline, and throughput) must be met concurrently. For example, with AI multi-tenancy, it is difficult to schedule AI tasks to meet their SLOs due to resource heterogeneity, as different edge devices and AI accelerators exhibit distinct performance characteristics and variations. A scheduler on edge devices should have accurate knowledge of the DL models’ performance, inference latency, and resource consumption when running with other DL models. However, having an accurate understanding of such characteristics is a complex task.

Moreover, while on-device model compression may generate a model that meets multiple SLOs, it faces two main obstacles. The first obstacle is the time and resource-intensive nature of on-device compression as the compression process demands substantial computing resources [19]. Given the constrained edge resources, this process often results in considerable scheduling delays. The second obstacle is the inference accuracy of compressed models. Due to the complexity of determining the right compression options [20], it is challenging to create a compressed model that effectively meets specific/complex SLOs.

For performance and energy efficiency, edge AI systems are often constrained by the rigid, ‘all-or-nothing’ execution of static deep learning models. This approach is highly inefficient, forcing the system to expend maximum computational effort and energy for every single inference, even when a user’s SLOs or the simplicity of the input data do not require the model’s full accuracy. To address this, dynamic inference techniques, which selectively execute only a subset of a model’s layers or blocks, have emerged as a promising approach to tailor resource usage to specific needs [21]. Early-exit or layer-skipping networks, in particular, are popular choices compatible with many common architectures.

Yet, the reliability of dynamic inference is contingent upon the intelligence of its control policy. Simply bypassing parts of a neural network can lead to unpredictable and often catastrophic drops in accuracy, making the inference result useless [22]. Consequently, dynamic systems must navigate a complex, non-linear trade-off space between computational savings (latency and energy) and model performance. These systems must decide, on a per-inference basis, which parts of the model are non-essential to meet the immediate SLOs.

Moreover, the optimal decision for this trade-off is not static; it is highly dependent on the input data itself. A skipping configuration that works well for a simple input may be insufficient for a more complex

one, making it challenging to guarantee performance across a diverse and unpredictable data stream [23]. This lack of a universally optimal configuration complicates real-time energy management and efficient SLO satisfaction, potentially causing frequent violations. Thus, intelligently managing a model’s internal computational graph in real-time to co-optimize for energy, performance, and accuracy remains a significant and open research challenge for edge AI systems.

This research proposal aims to address the above research challenges by developing a novel, end-to-end framework specifically tailored for edge AI inference serving systems. Our research focuses on building adaptive and energy-efficient systems designed to operate under the challenging conditions of strict power budgets, hardware heterogeneity, conflicting Service Level Objectives (SLOs), and dynamic, non-stationary workloads. Specifically, we aim to answer the following research questions (RQs).

- RQ1:** How can we design and implement an intelligent, multi-objective scheduling and coordination framework that holistically integrates heterogeneous accelerator management, AI multi-tenancy, and concurrent SLO satisfaction within edge environments, while dynamically adapting to workload fluctuations and model uncertainty to sustain optimal system-level efficiency and responsiveness?
- RQ2:** How can we design and implement a runtime adaptation system, comprising a model-aware online profiler and a reconfigurable inference execution engine, that uses dynamic layer bypassing in ML models while ensuring minimal computational overhead? Subsequently, what methodologies can be developed to make this system resilient to data distribution shifts, and how can the system efficiently select runtime configurations that satisfy multiple SLOs while minimizing adaptation overhead?
- RQ3:** How can we design and implement a deployment-feedback-driven self-calibration framework that continuously maintains adaptation effectiveness under evolving workload characteristics, resource availability, and non-stationary deployment environments? Furthermore, how can such a framework leverage historical deployment experience to autonomously detect adaptation degradation, refine decision policies, and sustain long-term multi-SLO satisfaction on resource-constrained edge devices?

To answer the RQs above, this proposal outlines three primary research thrusts as shown in Fig. 1.

Thrust #1 - This thrust first establishes the state-of-the-art in static scheduling through the design and implementation of Convergo, a framework engineered to address multi-SLO satisfaction in heterogeneous edge AI environments. The technical approach moves beyond simple prioritization heuristics by framing inference orchestration as a multi-stage, constraint-based optimization problem. The core of this thrust is a novel multi-objective scheduling algorithm that operates in two distinct phases. First, upon receiving an inference request with specific SLOs (e.g., accuracy > 90%, latency < 100ms, throughput = 5000), a candidate filtering stage consults a continuously updated performance database to prune the set of available model-accelerator pairs, immediately discarding any combination that cannot meet the request’s hard constraints. Second, a utility-based selection stage evaluates the remaining valid candidates, calculating a score for each based on how well it satisfies the soft constraints, such as the amount of temporal slack before the deadline. The algorithm then selects the model-accelerator pair with the highest utility score for execution, ensuring the "best fit" is chosen. Furthermore, to handle real-world uncertainty, we incorporated a slack-aware minimal profiling technique. This mechanism detects when an unprofiled model arrives and, if system resources are not under contention, triggers a brief in-situ profiling run to populate the performance database. This structured, data-driven approach creates a robust system that maximizes the concurrent satisfaction of complex SLOs for static, ‘black-box’ models with high efficiency and negligible overhead.

Thrust #2 - This thrust will fundamentally transcend the ‘black-box’ model paradigm by developing a novel co-design methodology for ML model architectures and their associated training regimes. The primary objective is to imbue models with intrinsic structural elasticity, making them inherently ro-

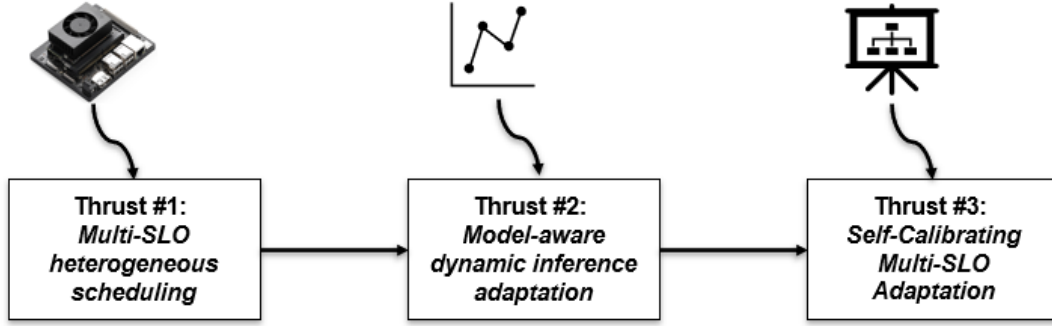


Figure 1: Overview of this project.

bust to the runtime bypassing of internal layers or blocks. This will involve investigating specialized training strategies, potentially including knowledge distillation and auxiliary loss functions, to ensure model stability. The ultimate goal of this thrust is to produce a new class of 'skip-aware' ML models whose performance-accuracy degradation landscape is stable, predictable, and quantifiable, thus providing the necessary foundation for fine-grained dynamic control.

Thrust #3 - Building upon the adaptation capabilities developed in the previous thrusts, we will design and implement a deployment-feedback-driven self-calibrating adaptation framework for edge AI systems. Unlike conventional adaptation approaches that assume stable operating conditions, the proposed framework continuously monitors deployment behavior, detects adaptation degradation caused by workload shifts, resource contention, and environmental variability, and autonomously refines its adaptation knowledge over time. The system integrates runtime monitoring, deployment feedback analysis, drift detection, and lightweight online calibration mechanisms to sustain long-term multi-SLO satisfaction under non-stationary deployment conditions. By transforming adaptation from a one-time optimization process into a continual self-improvement process, this thrust advances edge AI systems toward autonomous, resilient, and self-optimizing operation.

Proposal Structure. The rest of this proposal is organized as follows: Section 2 reviews related work; Section 3 presents the proposed research and evaluation plans in detail; Section 4 describes the preliminary work; and Section 5 provides the research timeline, summarizes this proposal, and discusses the contributions of this work.

2 Related Work

2.1 AI on Resource-Constrained Edge Devices

The challenge of on-device AI inference serving, which often struggles to meet demanding SLOs due to inherent resource constraints, has prompted the proposal of multiple facilitation approaches [24] [14] [12] [25] [15] [26]. One prevalent strategy, explored by Han *et al.*[26] and Lee *et al.*[14], involves employing efficient model compression techniques to substantially reduce the computational load on edge devices. While these methods are effective at shrinking model size and accelerating inference time, they frequently introduce a critical drawback: a degradation in performance when subjected to strict, multi-faceted SLOs, such as the simultaneous requirement of maintaining high accuracy while meeting a tight latency budget. In stark contrast, Convergo is engineered to perform a more sophisticated optimization, selecting models based on a holistic evaluation of multiple SLO constraints, thereby ensuring that target accuracy is preserved while

successfully meeting other objectives like throughput and deadlines. Another class of solutions, proposed by Xie *et al.*[12] and Zhang *et al.*[25], focuses on on-device inferencing frameworks that distribute computationally intensive workloads across a network of multiple devices. However, these distributed approaches often face significant challenges with high variability in both model accuracy and end-to-end latency, a problem exacerbated by the diverse and unpredictable nature of heterogeneous edge hardware.

Convergo directly tackles this issue by incorporating a heterogeneous scheduling engine that dynamically and intelligently selects the optimal models and accelerators for each individual task, effectively balancing the complex trade-offs between accuracy, latency, and throughput. Similarly, Wang *et al.*[15] introduced an adaptive inference system that dynamically compresses models based on real-time resource availability. While this system adeptly reduces latency, its singular focus on compression means it struggles to consistently meet multiple SLOs, particularly when latency and accuracy must be guaranteed together, due to the unavoidable accuracy drops induced by compression. In contrast, Convergo utilizes a fully multi-SLO-aware scheduling algorithm that prioritizes inference tasks and adapts to changing system conditions to ensure consistently high satisfaction across all defined SLOs.

On-device AI inference serving persistently struggles to satisfy stringent SLO requirements due to the fundamental resource constraints of edge hardware, a challenge that has led to multiple proposed solutions to facilitate on-device AI inference [24] [27] [12] [28] [29]. For instance, researchers like Liu *et al.*[24] and Lee *et al.*[27] have explored the application of efficient model compression techniques as a means to reduce the computational burden on edge devices. While these methods successfully shrink model footprints and shorten inference times, this efficiency often comes at the steep cost of significant performance degradation, particularly when operating under strict SLO constraints that demand the simultaneous satisfaction of multiple objectives. In contrast, Convergo is designed to overcome these very limitations by implementing an intelligent model selection process that is optimized based on a comprehensive set of multiple SLO constraints, ensuring that model accuracy is rigorously maintained while concurrently meeting other critical SLOs like throughput and deadlines. In a different vein, Xie *et al.*[12] and Zhang *et al.*[28] proposed on-device frameworks that distribute heavy workloads across multiple edge devices. However, these distributed approaches inherently suffer from high variability in both model accuracy and execution latency, largely because of the unmanaged heterogeneous nature of the underlying edge hardware. And [30] shows that leverage the optimized XLA compiler and low-precision float16 arithmetic to enable larger batch sizes and faster memory-bound operations, the TPU can often achieve significant speedups (up to 10x for RNNs) that exceed advances made using GPUs or CPUs for many deep learning workloads. Convergo directly addresses this core challenge by building its foundation on a heterogeneous scheduling engine, which dynamically selects the most suitable models and accelerators for every task, thereby achieving a carefully calibrated and balanced trade-off between accuracy, latency, and throughput. Furthermore, adaptive inference systems, such as the one introduced by Wang *et al.*[29] that dynamically compresses models based on current resource availability, can effectively reduce latency but lack the broader capability to consistently meet multiple SLOs simultaneously. Convergo, on the other hand, employs a far more sophisticated multi-SLO-aware scheduling algorithm that actively prioritizes inference tasks and fluidly adapts to changing conditions, guaranteeing a high degree of SLO satisfaction across a wide range of diverse operational scenarios.

2.2 Edge AI Schedulers

While the field has produced several edge AI schedulers designed to meet specific SLOs [9] [10] [2] [31] [1], a common and significant limitation is their tendency to focus on a single objective, often failing to address the multifaceted challenge of satisfying multiple, conflicting SLOs simultaneously. For instance, while SLO-MAEL [9] presents a sophisticated method for prioritizing tasks based on expected latency and the risk of

SLO violation, its design does not incorporate native mechanisms for managing heterogeneous hardware resources or balancing diverse multi-SLO requirements. This architectural omission ultimately leads to suboptimal performance when faced with complex application scenarios demanding concurrent guarantees on accuracy and throughput alongside latency. In direct contrast, Convergo is fundamentally designed to address these gaps by intelligently leveraging the distinct capabilities of heterogeneous AI accelerators to meet multiple SLOs concurrently. Similarly, schedulers like Kalmia [10] and Dělen [2] excel at optimizing for high inference throughput and efficient resource utilization; however, this focus frequently comes at the direct expense of other critical SLOs, such as maintaining model accuracy or adhering to strict deadlines. Convergo distinguishes itself by employing a holistic scheduling algorithm that actively balances these conflicting requirements, thereby ensuring that all user-defined SLOs are met without disproportionately compromising overall system performance. Furthermore, while systems like Clipper [31] and Masa [1] are tailored for low-latency prediction serving and multi-DNN inference, their frameworks exhibit a lack of flexibility when confronted with dynamic, multi-SLO requirements in heterogeneous edge environments. The core design of Convergo, with its integral support for multi-tenancy and dynamic model selection, provides the necessary adaptability to optimize for multiple SLOs simultaneously, proving effective even within highly resource-constrained settings.

Although there are several edge AI schedulers designed to meet SLOs [9] [10] [2] [31] [1], their primary limitation stems from a design philosophy that does not inherently account for the need to satisfy multiple, often contradictory, SLOs simultaneously. A clear example of this is SLO-MAEL [9], an edge AI scheduler engineered to prioritize tasks based on predicted latency and SLO violation risks. While this approach is highly effective in minimizing latency-related violations for time-sensitive tasks, SLO-MAEL lacks a comprehensive, integrated mechanism for managing heterogeneous compute resources or navigating the complex trade-offs posed by multi-SLO requirements, which ultimately leads to suboptimal performance when applications demand more than just low latency. To address these very gaps, Convergo incorporates a sophisticated scheduling engine that explicitly leverages heterogeneous AI accelerators, enabling it to meet a diverse set of SLOs concurrently. Other schedulers, such as Kalmia [10] and Dělen [2], are purpose-built to optimize inference throughput and maximize resource utilization on edge servers. Although these systems excel in their designated goal of maximizing computational efficiency, this often occurs at the significant expense of other critical SLOs, including the preservation of model accuracy and strict adherence to task deadlines. In stark contrast, the scheduling algorithm within Convergo is engineered to meticulously balance these conflicting requirements, ensuring that no single SLO is sacrificed and that overall system performance remains robust. Finally, inference serving systems like Clipper [31] and Masa [1], which concentrate on low-latency prediction serving and multi-DNN inference, tend to lack the intrinsic flexibility required to handle complex multi-SLO requirements, particularly within the dynamic context of heterogeneous edge environments. The foundational multi-tenancy and adaptive model selection capabilities of Convergo allow it to vastly outperform such systems by simultaneously optimizing for multiple SLOs, demonstrating its efficacy even under stringent resource constraints.

3 Proposed Research

The objective of this research is to develop a unified edge-AI framework that enables intelligent multi-SLO scheduling, model-aware runtime adaptation, and intelligent scheduling, model-aware adaptation, and deployment-feedback-driven self-calibration, achieving an edge computing ecosystem that is autonomous, energy-efficient, performance-optimized, and resilient under dynamic workloads and resource constraints.

3.1 Thrust I: Multi-SLO heterogeneous scheduling

AI inference is increasingly executed on edge devices, which must simultaneously satisfy multiple, complex, and often conflicting Service Level Objectives (SLOs) like accuracy, throughput, and latency. This challenge is compounded by the growing heterogeneity of on-device AI accelerators (e.g., GPUs, TPUs), each with unique performance characteristics. Therefore, a critical step is to move beyond simple task-offloading and develop an intelligent coordinator that can holistically orchestrate these diverse resources. Existing scheduling works are often insufficient, as they tend to focus on a single SLO or fail to fully leverage resource heterogeneity in a multi-tenant environment. Thus, this part of the research will focus on how to design a novel, multi-objective scheduling framework that can treat inference orchestration as an optimization problem, maximizing the concurrent satisfaction of all user-defined SLOs.

3.1.1 Proposed Approach of Thrust 1

Convergo is a novel AI scheduling framework that integrates heterogeneous accelerator management, multi-tenancy, and multi-SLO prioritization into a unified solution for edge devices. It uses scheduling heuristics to prioritize inference requests, enforce critical constraints, and dynamically select the best model-accelerator combinations to simultaneously satisfy multiple service level objectives, including accuracy, throughput, and latency. Evaluated on a Jetson Xavier platform with portable TPUs, the system outperforms existing baselines, achieving over 90% simultaneous SLOs satisfaction and approximately 95% individual SLOs satisfaction with negligible overhead.

This section describes edge devices and AI accelerators, DL models, and DL frameworks used in this study.

Edge devices. The prototype of Convergo has been implemented on Nvidia’s Jetson Xavier [32], which is equipped with multiple Coral TPU accelerators [33]. Jetson Xavier is an edge device equipped with a 6-core Carmel CPU, 8GB of RAM, and a Volta GPU with 386 Tensor Cores for AI operations. Coral TPU accelerator is a portable AI accelerator, delivering 4 TOPS at 2W of power consumption.

DL Models. To evaluate Convergo’s performance, we employed three edge AI applications: *image classification*, *object detection*, and *pose estimation*. These applications use a variety of pre-trained models listed in Table 1. Specifically, image classification involves eight models, with four tailored for GPU and four for TPU inferences. Even when models share the same architecture (e.g., Inception-v1 [34] for both GPU and TPU in the image classification category), they are individually optimized for each type of accelerator. For object detection and pose estimation, we use three GPU models and three TPU models for each, respectively.

Workload Generation. Our evaluation workloads combine all three above AI applications, with each request being randomly generated to assess Convergo under more realistic use cases. The evaluation workloads consist of a set of 500 requests generated using the Poisson distribution. Each request within these workloads has specific accuracy, throughput, and latency (deadline) requirements. Moreover, we conducted the evaluation with multiple iterations with 10 different workloads to mitigate the impact of outlier results.

Baselines. To evaluate the performance of Convergo, we employed two state-of-the-art baselines: SLO-MAEL [9] and Kalmia [10]. SLO-MAEL is built upon a minimum-average-expected-latency (MAEL) scheduling algorithm by integrating SLO awareness into its operations. SLO-MAEL uses a dual-scoring mechanism where tasks are evaluated based on both expected latency and their adherence to SLOs. The scheduler prioritizes tasks that are most likely to violate their SLOs, trying to ensure that critical tasks are processed within their required deadlines. Additionally, SLO-MAEL incorporates preemptive scheduling, allowing it to interrupt and reprioritize ongoing tasks based on the degree of SLO violation. Kalmia [10] is a QoS-aware scheduler designed to manage heterogeneous DNN inference tasks on edge servers via a mix of

Table 1: AI Applications and DL Models Used For Evaluation.

AI Application	Edge Acceler.	Pre-trained DL Models	Size (MB)	Num. Layers
Image Classification	GPU	EfficientNet [35]	5.4	5
		Inception-v1 [34]	6.4	22
		MobileNet-v2 [36]	3.4	20
		SqueezeNet [37]	1.3	15
	TPU	EfficientNet	6.8	7
		Inception-v1	7.0	22
		Inception-v3 [38]	12.0	48
		MobileNet-v1 [39]	1.6	28
Image Classification	GPU	EfficientNet [35]	5.4	5
		MobileNet-v2	6.5	20
		YOLOX [40]	5.4	11
	TPU	EfficientNet	7.6	18
		MobileNet-v1	7.0	28
		MobileNet-v2	6.6	20
Image Classification	GPU	EfficientNet [35]	5.4	5
		MoveNet [41]	5.6	25
		PoseNet [42]	4.3	23
	TPU	MoveNet	7.5	25
		PoseNet-MobileNet [42]	1.5	23
		PoseNet-ResNet [42]	24.4	18

preemptive and context-aware scheduling strategies to optimize both task timeliness and system throughput. AI tasks are categorized into urgent and non-urgent types. For urgent tasks, it enables preemptive scheduling to give them higher processing priority, while non-urgent tasks are used to increase resource utilization by leveraging slack time.

3.1.2 Evaluation Plan

To assess its performance, we conducted extensive evaluations across three widely-used AI application categories: image classification, object detection, and pose estimation. We generated realistic AI inference workloads, with each request having three distinct SLO requirements: accuracy, throughput, and inference deadline. Convergo leverages multiple pre-trained models and must select candidate models that fulfill all the requirements. In our design, we assume that multiple users submit various requests to Convergo. A user request refers to an AI inference task submitted by users, with each request having its own multiple SLOs that need to be satisfied simultaneously. For example, each request (R_i) contains three specific sub-requirements, denoted as $\langle Th_i, Acc_i, D_i \rangle$, where Th_i represents the throughput goal, Acc_i is the accuracy target, and D_i is the deadline of the request (R_i).

To quantify the performance, Convergo creates three constraints (C_1 , C_2 , and C_3), The first constraint

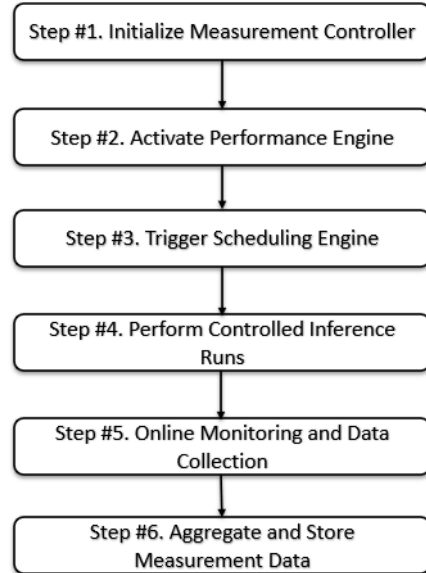


Figure 2: Measurement steps

(C_1) is the throughput requirement of R_i , and candidate models will be chosen based on this equation.

$$\sum_{j=1}^n \mathcal{M}_j(Th) \times t_j \geq Th_i, \quad (1)$$

where $\mathcal{M}_j(Th)$ represents the throughput statistics of j^{th} model ($\mathcal{M}_j$), t_j denotes the execution time of $\mathcal{M}_j$. Given the models that can meet the throughput requirement, Convergo uses the second constraint (C_2) to select models that meet the accuracy requirement. Equation-(2) is used to calculate the overall model accuracy, and only models with an overall accuracy higher than A_j will be selected. In equation-(2), $\mathcal{M}_j(Acc)$ represents the accuracy of $\mathcal{M}_j$.

$$\frac{\sum_{j=1}^n \mathcal{M}_j(Th) \times t_j \times \mathcal{M}_j(Acc)}{\sum_{j=1}^n \mathcal{M}_j(Th)} \geq Acc_i, \quad (2)$$

Given the model selected for throughput and accuracy requirements, Convergo now needs to apply the third constraint (C_3) to determine the models that can satisfy all the requirements. This step tests whether the models can meet the deadline (D_i) requirements by calculating

$$\sum_{j=1}^n t_j \leq D_i. \quad (3)$$

By using these three constraints, Convergo identifies the candidate models ($\mathcal{M}_C$) that can meet R_i 's SLOs.

Measurement Procedure. We will develop a benchmarker to evaluate the Convergo framework and measure deep learning inference performance, SLO satisfaction, and system statistics on heterogeneous edge devices. The benchmarker will be deployed with representative DL applications—image classification, object detection, and pose estimation, across the Jetson Xavier platform and portable TPU accelerators. The measurement procedure, illustrated in Fig. 2, proceeds as follows.

The scheduler begins by initializing the Measurement Controller (step #1), where experimental parameters including DL model information, target accelerators, user requirements and SLO configurations (accuracy, latency, throughput) are specified. It then activates the Performance Engine (step #2) to gather and update the performance database with model, and system-level characteristics such as inference time and resource utilization. Next, the Scheduling Engine is triggered (step #3) to generate scheduling decisions, selecting the most suitable models, and accelerator combinations to satisfy multiple concurrent SLOs. Once the scheduling plan is established, the system performs controlled inference runs (step #5) according to the assigned configuration. During these runs, the online monitoring module (step #6) continuously collects runtime statistics including latency, throughput, and power usage, and feeds them back into the performance database to refine subsequent scheduling cycles. Finally, we aggregate and store all measurement data (step #7) for comprehensive analysis of Convergo's behavior, allowing empirical evaluation of its ability to maximize SLO satisfaction, leverage heterogeneous accelerators, and maintain lightweight operation under realistic edge workloads.

System Statistics and Power Measurement. In the above measurement step, diverse scheduler statistics will be collected while the inference tasks are being performed. `sysstat` [43] will be used to collect the

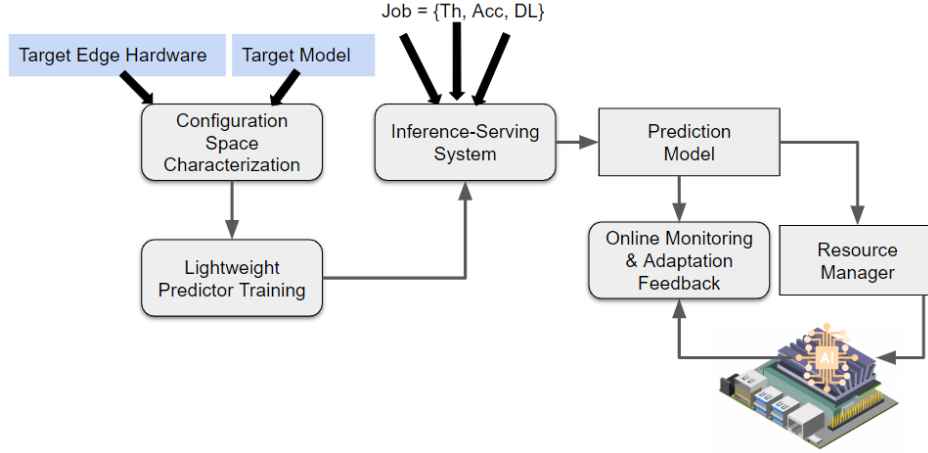


Figure 3: **Overview of Thrust 2.**

usage of CPU, memory, and Disk/USB IO. For measuring the power consumption of edge devices, we use `jetson-stats` [44] to measure the power consumption of Jetson Xavier. For EdgeTPUs, we use a USB power meter.

3.2 Thrust II: Model-aware dynamic inference adaptation

Fig. 3 illustrates the end-to-end workflow of Thrust 2 as a two-stage pipeline that separates offline model preparation from online use. In the offline stage, the system enumerates a set of allowable skip configurations (e.g., at block boundaries), trains the model to remain accurate under those configurations, and then profiles each configuration to build an elasticity profile that maps configuration to expected SLOs. In the online stage, each incoming inference request consults this pre-built profile to select a suitable skip configuration, executes inference using a reconfigurable engine that activates only the chosen blocks, and records runtime outcomes to enable validation and future refinement of the offline profile or training recipe.

3.2.1 Proposed Approach of Thrust 2

The widespread adoption of edge AI is fundamentally hampered by the inherent inefficiency of static inference models, which demand full execution of all layers and blocks for every single inference. This rigid, full-execution approach is ill-suited for resource-constrained edge environments, leading to excessive energy consumption and computational load, particularly when user Service Level Objectives (SLOs) do not require the model’s maximum possible accuracy. Existing systems lack the mechanisms for runtime adaptation of both the model’s computational graph and the underlying system parameters. Thus, we design a skip-aware neural network whose internal blocks can be bypassed at runtime without causing unstable or catastrophic accuracy drops. The key outcome is not a new scheduler, but a model-side “elasticity contract”, it is a small, well-defined set of allowed skip configurations (e.g., skipping at specific block boundaries) with predictable accuracy behavior. These models provide the necessary foundation for later runtime control by ensuring that skipping more computation results in a stable and quantifiable reduction in accuracy rather than an unpredictable failure mode.

Characterizing the Configuration Space. To enable intelligent runtime decision-making, the system must first understand the complex trade-offs between performance, accuracy, and energy across a vast configuration space. This space is defined by a combination of model-level and system-level control knobs. For this

project, we characterize this space on target edge hardware, such as the NVIDIA Jetson Orin Nano [32]. The primary control knobs include:

First, at the model level, it involves dynamically bypassing non-essential layers or blocks within the ML model. For a model like ResNet-34 (Skip-Net[22]), this creates 4,095 unique execution paths or skip-configurations. And at the system level, these knobs control the hardware execution environment. They include batch size (e.g., discrete values from 1 to 20, such as 1, 2, 4, 8, 16) and power mode. An Orin Nano offers 4 power modes, which, when combined with `jetson_clocks` (turn it on or off), yield 8 distinct power settings. The combination of these knobs results in a combinatorial explosion. For a single model like ResNet-34 on an Orin Nano, this creates a search space of over 160,000 unique configurations. Exhaustively profiling this entire space is impractical. Therefore, our methodology involves sampling a strategic subset (approx. 10,000 points) of these configurations to collect a training dataset. For each sampled point, we will measure the resulting inference accuracy, latency, and power consumption, with power being measured via onboard sensors or external measurement tools.

We will make the skipped paths accurate and consistent, and we will train the model to be robust under skipping via multi-path training, where each mini-batch samples one or more allowed skip configurations and updates shared weights. Concretely, the training objective will combine the task loss for each sampled path and optional consistency regularization, so neighboring configurations produce smoothly varying outputs.

Performance and Accuracy Prediction. Searching the massive configuration space at runtime to find the optimal operating point for a given SLO is computationally infeasible. The core of our system is a set of lightweight prediction models that can accurately and rapidly forecast the outcome of any given configuration without the overhead of actual execution. This approach decouples the search for an optimal configuration from the execution itself. We propose a two-model strategy to efficiently predict performance and accuracy.

Service Level Objectives (SLOs) are user-defined goals for metrics such as inference latency, throughput, and accuracy. Our prediction models are designed to map configuration parameters to these metrics.

To determine the optimal configuration, we must predict the latency and accuracy for different combinations of our control knobs. We will develop two specialized, lightweight predictors:

Accuracy Prediction Model: This model’s sole function is to predict the inference accuracy (e.g., top-1 or top-5) for a given model-level skip-configuration. A key insight is that accuracy is independent of system-level knobs like batch size and power mode. This specialization simplifies the model, for which we may explore techniques from approximate computing. **Latency Prediction Model:** This model predicts the inference latency given a full configuration vector: the skip-configuration, batch size, and power mode. To ensure minimal overhead, we will implement this as a lightweight model, such as a small Convolutional Neural Network (CNN). The model will be trained on the collected profiling data using approximately 20 input features, including the layer-skip vector, CPU/GPU frequencies, selected power mode, and batch size. By using these predictors, the system’s Resource Manager (Scheduler) can efficiently evaluate thousands of potential configurations in milliseconds, identify the one that best meets the user’s SLOs while minimizing energy, and dispatch it for execution on the edge device.

During execution, an online monitor watches latency drift, accuracy proxies, and power consumption. If violations occur, the controller performs fast adaptations such as:

- Tighten/loosen skip mask
- Adjust batch size to trade throughput for latency
- Escalate power mode temporarily under constraints

All adaptations and outcomes are logged and fed back to retrain predictors periodically.

3.2.2 Evaluation Plan

Our evaluation aims to assess the performance of our prediction models and the overall efficacy of the dynamic inference serving system. The accuracy of the latency and accuracy prediction models is critical for our system, as it enables the scheduler to find optimal configurations that satisfy user SLOs without the need for exhaustive, real-time profiling. We will first measure the accuracy of our prediction models, reporting metrics such as prediction error (RMSE/MAPE) between predicted and observed values. And we will compare our work with nnMeter [45], and Delen [46].

The core of the evaluation will focus on the performance of the complete dynamic scheduling system under realistic workloads. To rigorously quantify the benefits of our approach, we will compare the performance of our dynamic serving system against three key baselines.

Static Full-Model: A conventional approach where the full model is always executed at a fixed, high-performance power mode, representing a performance-first strategy. **Batch and Power Tuning Only:** A system that dynamically tunes only batch size and power modes but does not perform any model-level skipping. This isolates the gains from system-level adaptation. The fixed skip configuration will be a system using a single, pre-optimized "skipped" model version without the ability to adapt the skip configuration online. This demonstrates the value of dynamic model adaptation. The evaluation will measure a comprehensive set of key metrics, including SLO satisfaction rate, energy consumption per inference, the delta between achieved and target accuracy, and tail latency (p95/p99). We will also measure the adaptation overhead to ensure our decision-making process is lightweight and efficient.

Finally, to understand the individual contributions of our system's components, we will conduct a thorough ablation study. This study will isolate the performance impact of each control knob (model skipping, batch sizing, and power management). We will also analyze the effect of predictor error on the scheduler's robustness, investigating how error bounds might necessitate safety margins in decision-making to guarantee SLO compliance.

Evaluation Dataset and Workload. We will generate a comprehensive dataset by profiling various ML models, including ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152 [47] for image classification on target edge hardware, including NVIDIA Jetson Orin Nano, NVIDIA Jetson AGX Xavier [32], GeForce RTX 2080 Ti [48], and RTX A5000 [49]. This dataset will map thousands of unique configurations, their combinations of skip-configurations, batch sizes, and power modes to their resulting performance metrics (latency, accuracy, power consumption). For the final performance evaluation, we will create a workload comprising a sequence of inference jobs, each with distinct SLOs for latency and accuracy. It is critical to note that the configuration data used to train our prediction models will be strictly separated from the configuration space and job workloads used for the final system performance evaluation, ensuring a fair and unbiased assessment.

For each trained skip-aware model, we will systematically characterize its performance-accuracy degradation surface across thousands of possible skipping configurations. This will produce a compact, runtime-usable model of expected accuracy, latency, and energy as a function of which layers are skipped. The resulting degradation landscape will be stable and predictable, allowing the runtime system to treat skipping decisions as safe, quantifiable trade-offs.

3.3 Thrust III: Self-Calibrating Multi-SLO Adaptation

While Thrust 2 enables predictor-guided dynamic adaptation by characterizing the performance trade-offs of elastic models and training lightweight prediction models, it implicitly assumes that these predictors remain accurate after deployment. In practice, this assumption rarely holds in long-term edge deployments. Edge devices operate under highly dynamic conditions, including workload fluctuations, thermal throttling, resource contention, hardware aging, software updates, and changing input data distributions. These factors can gradually alter the relationship between configuration choices and observed performance, causing prediction drift and reducing the effectiveness of adaptation decisions.

As a result, configurations that were previously optimal may no longer satisfy latency, accuracy, or energy objectives. In addition, a runtime adaptation framework that relies on static profiling and offline-trained predictors may experience increasing SLO violations over time. This challenge motivates this thrust of this dissertation: the development of a deployment-feedback-driven self-calibrating adaptation framework capable of maintaining adaptation effectiveness throughout long-term deployment.

Unlike Thrust 2, which focuses on predicting the performance of candidate configurations, Thrust 3 focuses on preserving the validity of those predictions after deployment. The central research question is: How can edge AI systems continuously maintain multi-SLO adaptation effectiveness under evolving workload and device conditions? To answer this question, we propose a Self-Calibrating Adaptation framework that continuously monitors deployment outcomes, detects prediction drift, and updates adaptation knowledge using runtime feedback. Rather than retraining the underlying neural network, the framework incrementally refines its adaptation models and decision policies to remain effective under changing operating conditions.

3.3.1 Proposed Features for Thrust 3

We will develop a deployment-feedback-driven adaptation framework that continuously evaluates and improves its runtime behavior throughout long-term deployment. The proposed system combines runtime monitoring, feedback analysis, drift detection, and self-calibration mechanisms to create a self-improving edge AI platform capable of sustaining multi-SLO performance under evolving operating conditions.

The framework maintains utility scores for candidate configurations based on deployment feedback. When deployment drift is detected, configuration rankings are adjusted to reflect observed performance rather than relying solely on historical estimates. This allows the system to gradually favor configurations that remain effective under current operating conditions.

A key feature of the proposed framework is its ability to detect adaptation degradation. By comparing expected system behavior with observed runtime outcomes, the system can identify emerging performance drift caused by workload changes, thermal effects, resource contention, or environmental variability. Once degradation is detected, the framework automatically initiates a calibration process to refine adaptation knowledge and update runtime decision policies.

The proposed system also incorporates long-term adaptation memory. Rather than relying solely on instantaneous observations, the framework leverages historical deployment experience to recognize recurring workload patterns and recurring system states. Historical observations are maintained using a sliding-window mechanism whose size will be evaluated experimentally. This capability allows the system to make increasingly informed decisions as deployment duration increases.

Another important feature is lightweight runtime operation. Since edge platforms possess limited computational resources, all monitoring, calibration, and adaptation mechanisms must incur minimal overhead. The framework emphasizes efficient online learning techniques that can operate continuously on resource-constrained devices without disrupting inference performance.

Together, these features transform adaptation from a one-time optimization process into a continual self-improvement process capable of sustaining system performance throughout extended deployment lifetimes.

3.3.2 Proposed Approach for Thrust 3

The proposed approach introduces a deployment-feedback-driven self-calibration framework that continuously evaluates and improves adaptation effectiveness throughout long-term deployment. Unlike traditional adaptation systems that assume a stable operating environment, the proposed framework treats deployment as a non-stationary process in which workload characteristics, hardware behavior, resource availability, and application requirements evolve over time. The primary objective is to maintain multi-SLO satisfaction despite these changes by enabling the runtime system to continuously learn from deployment experience.

The proposed architecture consists of five tightly coupled components: (1) Runtime Monitoring Engine, (2) Deployment Feedback Repository, (3) Drift Detection Module, (4) Self-Calibration Engine, and (5) Adaptive Decision Controller. These components form a closed-loop adaptation cycle that continuously observes system behavior, analyzes deployment outcomes, updates adaptation knowledge, and refines future decisions. During initial deployment, the system falls back to the predictor-guided adaptation framework developed in Thrust 2. Self-calibration is activated only after sufficient deployment feedback has been collected.

The Runtime Monitoring Engine serves as the system’s observation layer. During execution, it continuously collects workload characteristics, request arrival patterns, hardware utilization, power consumption, latency measurements, throughput statistics, and application-level performance metrics. Unlike traditional profilers that focus solely on instantaneous measurements, the monitoring engine captures both short-term fluctuations and long-term behavioral trends. This information provides the foundation for understanding how system performance evolves throughout deployment.

The Drift Detection Module periodically analyzes the deployment feedback repository to evaluate adaptation effectiveness. The primary purpose of this module is to identify situations in which current adaptation strategies are no longer producing expected results. Such situations may arise from thermal throttling, hardware aging, workload shifts, resource contention, software updates, or changes in input data characteristics. By continuously comparing expected outcomes with observed runtime behavior, the system can identify emerging performance drift before substantial SLO degradation occurs. Drift is detected when the prediction error exceeds a configurable threshold over a sustained observation window.

Upon drift detection, the Self-Calibration Engine analyzes recent deployment outcomes and computes updated utility scores for candidate configurations. Configurations that repeatedly satisfy multi-SLO requirements receive higher utility values, while configurations associated with frequent violations are penalized. The Adaptive Decision Controller uses these updated scores to dynamically re-rank candidate configurations and guide future adaptation decisions. Through repeated observation and feedback, the system continuously refines its configuration preferences without requiring expensive model retraining. The calibration engine maintains utility scores for candidate configurations. When drift is detected, utility scores are updated using recent deployment outcomes, and candidate configurations are re-ranked accordingly.

The Adaptive Decision Controller represents the action layer of the framework. Using information provided by the calibration engine, the controller selects adaptation actions that maximize long-term multi-SLO satisfaction. These actions may include selecting alternative execution configurations, adjusting resource allocations, modifying adaptation thresholds, or applying workload-specific optimization strategies. As calibration knowledge improves, the quality of these decisions also improves, enabling the system to gradually become more effective throughout deployment. The interaction among these components creates a continual learning loop. Runtime observations are first collected by the monitoring engine and stored in the deploy-

ment repository. The drift detection module then evaluates whether adaptation quality is deteriorating. When drift is detected, the calibration engine updates adaptation knowledge using deployment feedback. Finally, the adaptive controller applies the updated knowledge to future decisions, generating new deployment experiences that are again fed back into the monitoring system. Through repeated iterations of this cycle, the framework continuously improves its ability to maintain multi-SLO satisfaction under changing deployment conditions.

The proposed approach draws inspiration from several research directions in adaptive edge computing and online learning. [50] demonstrates the effectiveness of online learning under uncertain network and resource conditions. The system continuously updates offloading strategies based on observed rewards, illustrating how runtime feedback can improve decision quality without requiring complete system knowledge. However, its primary focus is task placement rather than maintaining adaptation effectiveness within AI inference systems. EdgeTimer [51] introduces a hierarchical scheduling framework that combines short-term and long-term decision making. The system demonstrates that multi-timescale adaptation can effectively respond to workload variability while maintaining scheduling efficiency. Although EdgeTimer focuses on scheduling and resource management, its use of deployment history highlights the importance of leveraging long-term operational information for adaptive decision making. [52] extends online learning by incorporating contextual information and non-stationary environment modeling. The framework demonstrates improved adaptation under changing operating conditions and emphasizes the importance of continual learning in heterogeneous edge systems. Nevertheless, the work primarily focuses on server selection and task allocation rather than maintaining long-term adaptation quality for AI inference workloads. And several additional studies provide valuable insights into deployment-aware adaptation. Recent edge-AI systems such as Ekya [53] demonstrate the importance of continuous adaptation and model maintenance in long-running edge deployments. Ekya focuses on continual retraining for video analytics pipelines, showing that deployment conditions can substantially alter model effectiveness over time. Also, Chameleon [54] highlights the benefits of adaptive configuration selection under changing operating environments.

Despite these advances, existing approaches largely focus on scheduling, resource allocation, task offloading, or application-specific retraining. None directly address the problem of maintaining adaptation effectiveness for multi-SLO edge AI systems through deployment-feedback-driven self-calibration. The proposed research differs by treating adaptation quality itself as the optimization target. Rather than merely selecting better actions, the framework continuously evaluates and improves the adaptation process, enabling sustained performance under long-term deployment drift.

3.3.3 Evaluation Plan

To evaluate this system, the following experiments are proposed:

- **Experiment design** - Experiments will be conducted on multiple edge platforms, including NVIDIA Jetson Orin Nano, NVIDIA Jetson AGX Xavier, GeForce RTX 2080 Ti, and RTX A5000. The system will be deployed continuously under realistic workload conditions to emulate long-term edge-AI operation. To evaluate robustness under non-stationary environments, we will intentionally introduce workload shifts, thermal throttling, resource contention, background processes, and changing power conditions throughout deployment. Representative AI workloads such as image classification and object detection will be used. The proposed framework will be compared against EdgeTimer, and Contextual Multi-Armed Bandits (CMAB).
- **Data to collect** - During deployment, we will continuously collect workload statistics, resource utilization, latency, throughput, energy consumption, adaptation decisions, and runtime monitoring information. In addition, deployment feedback records, drift detection events, calibration updates, and

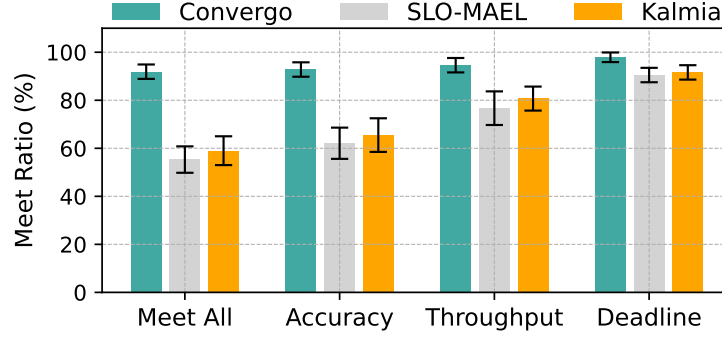


Figure 4: **SLO Satisfaction Results.** “Meet All” indicates the results when all three requirements are met. The other three – “Accuracy,” “Throughput,” and “Deadline” – represent the satisfaction rate for each individual requirement.

adaptation recovery behavior will be logged. For each adaptation cycle, we will record the observed system state, selected adaptation action, resulting performance metrics, and calibration outcomes.

- **Metrics** - The primary evaluation metrics will be Multi-SLO Satisfaction Rate, SLO Violation Rate, Adaptation Recovery Time, Calibration Convergence Time, and Prediction Drift Reduction. We will also measure long-term performance stability, runtime overhead, latency, throughput, and energy consumption. To evaluate self-calibration effectiveness, we will quantify the reduction in performance degradation before and after calibration events. Runtime overhead will be reported using CPU utilization, memory footprint, additional power consumption, and calibration latency. Furthermore, we will measure how quickly the system recovers from workload shifts and environmental changes compared to competing approaches.
- **Anticipated results** - We expect the proposed framework to maintain significantly higher Multi-SLO Satisfaction Rates during long-term deployment compared to existing adaptive systems. Following workload shifts or environmental changes, the framework may initially experience temporary performance degradation; however, through deployment-feedback-driven self-calibration, we expect the system to gradually recover and restore adaptation effectiveness. We further anticipate that the framework will reduce SLO violations, improve long-term stability, and sustain adaptation quality despite changing deployment conditions. Overall, the proposed approach is expected to demonstrate that Self-Calibrating Adaptation can provide more reliable and resilient edge-AI operation than static adaptation policies or conventional online learning approaches.

4 Preliminary Work

4.1 Thrust I: Multi-SLO heterogeneous scheduling

We now report the performance evaluation results of Convergo and the baselines.

SLO Satisfaction. Fig. 4 shows the SLO satisfaction results of Convergo and the baselines. Each request has distinct accuracy, throughput, and deadline requirements. We measured: (1) “Meet All”, which is the success rate when all requirements were met; and (2) the satisfaction rate for each individual requirement (e.g., “Accuracy,” “Throughput,” and “Deadline”, which are also shown in the graph).

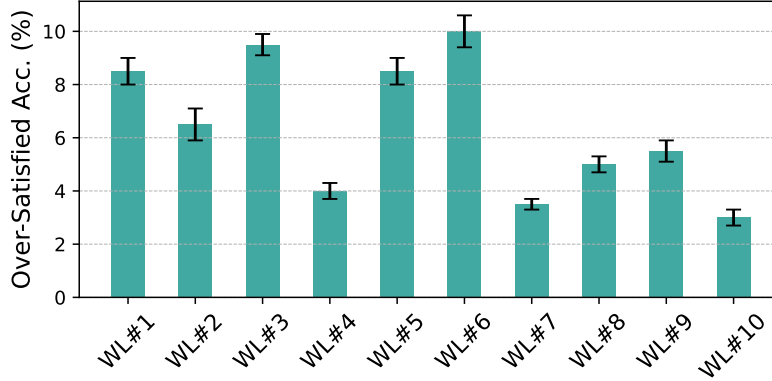


Figure 5: **Over-Satisfied Inference Accuracy (%) for All 10 Evaluated Workloads.** For example, 8.3% of WL #1 means that Convergo’s accuracy achievement is, on average, 8.3% higher than the requirements

Convergo achieved over 91% SLO satisfaction by meeting all three requirements simultaneously, which is significantly higher than the other baselines. Additionally, Convergo outperformed the other baselines by achieving high satisfaction rates for each individual requirement: 93% for accuracy, 95% for throughput, and 98% for deadlines met. Such a high SLO satisfaction rate is primarily due to the efficacy of Convergo’s scheduling algorithm, which enables the simultaneous execution of multiple AI models on different AI accelerators (e.g., models on both GPUs and TPU accelerators). Convergo’s multi-constraint design highlights its novelty relative to simpler solutions that focus on a single metric. By contrast, the baselines are primarily designed to meet either deadline or throughput requirements, missing many multi-SLO considerations. Hence, the baselines’ satisfaction rate for “Meet All” is less than 60% in Fig. 4. Additionally, they showed a satisfaction rate of less than 70% for the accuracy requirement and less than 80% for throughput requirements.

Over-Satisfied Inference Accuracy. We also measured the over-satisfied accuracy (defined as how much the actual accuracy surpassed the requirement) to demonstrate Convergo’s ability to maximize accuracy while meeting throughput and deadline requirements. As shown in Fig. 5, on average, Convergo provided 7% higher accuracy than required across the 10 evaluated workloads. We observed that Convergo provided over 8% over-satisfied accuracy for 4 of the evaluated workloads. These results confirm that Convergo not only achieves a high SLO satisfaction rate but also exceeds accuracy requirements in many cases, highlighting its additional ability to provide better quality of service (QoS).

Analysis of SLO-missed Cases. We analyzed how close the system came to meeting SLOs when they were missed. For the accuracy miss cases, Convergo’s results were only 1.3% below target accuracy; in contrast, the baselines were about 8% lower, confirming that Convergo’s accuracy achievements are much closer to its target accuracy. For the deadline miss cases, Convergo processed the missed jobs using over 58% of the request deadlines. However, SLO-MAEL processed the requests with 97% extra time over the deadline, and Kalmia used 134% extra time over the deadlines. These results showed that Convergo’s QoS degradation due to the missed accuracy requirement is negligible, and Convergo was able to complete the requests much earlier than the other baselines (even when the deadlines were not met).

However, the results measuring the gap between the achieved throughput and the target throughput (when the throughput requirements were not met) showed similar results in all three approaches. All three approaches showed 18% to 20% of missed throughput for the error cases.

Resource Utilization of Edge Devices. Fig. 6 reports the resource utilization and power consumption of the Jetson Xavier device for Convergo versus the baselines. In particular, Convergo increased CPU

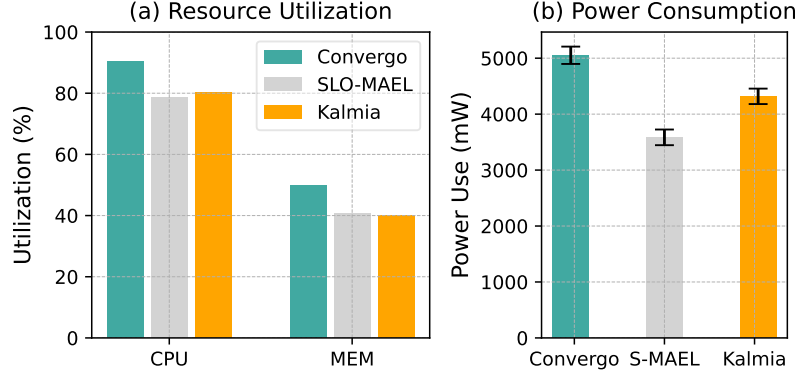


Figure 6: **Resource Utilization and Power Consumption.**

utilization by up to 12% and memory utilization by up to 10% over the baselines. This increased utilization is mainly due to Convergo’s sophisticated scheduling operations, which improved throughput and met SLO requirements. However, with respect to power consumption, Convergo consumed 700mW to 1500mW more power compared to the baselines. Despite Convergo’s significant improvements in other metrics (*e.g.*, multi-SLO satisfactions), its scheduling algorithm is more complicated and requires extra optimizations and operations that can consume more power. Therefore, improving power efficiency will be our next step in enhancing Convergo.

5 Timeline, Summary, and Research Contributions

5.1 Timeline

Table 2 shows the timeline of this research. Currently, I am conducting research on thrust 2 in order to enhance the skipnet scheduler, along with my previous work published in IEEE EDGE 2025 [55]. I also initiated thrust 3 and current efforts include studying non-stationary edge environments, runtime monitoring techniques, online learning methods, and self-calibration strategies for maintaining multi-SLO satisfaction under evolving workload and device conditions.

Semester:		1	2	3	4	5
Research	Thrust 1	████████	████████			
	Thrust 2		████	████████	████	
	Thrust 3				████████	████████
Paper	Paper Submission		■			■
Thesis	Thesis Writing & Defense					████████

Table 2: **Project Timeline**

5.2 Summary and Research Contributions

This research investigates efficient and reliable edge AI systems through three complementary research thrusts. Thrust 1 develops a multi-SLO scheduling framework for coordinating heterogeneous edge accelerators while simultaneously satisfying accuracy, throughput, and deadline requirements. Thrust 2 intro-

duces model-aware adaptation through elastic neural architectures and lightweight performance predictors that enable efficient exploration of large configuration spaces under dynamic runtime conditions. Thrust 3 develops a deployment-feedback-driven self-calibrating adaptation framework that continuously detects deployment drift, refines adaptation behavior using runtime feedback, and maintains long-term multi-SLO satisfaction under evolving operating conditions. Together, these contributions advance edge AI from intelligent scheduling to model-aware adaptation and ultimately to self-calibrating deployment intelligence for dynamic edge environments.

This project advances edge AI systems through three complementary contributions. First, it develops a multi-SLO scheduling framework capable of coordinating heterogeneous edge accelerators while simultaneously satisfying accuracy, throughput, and deadline requirements. Second, it introduces model-aware adaptation mechanisms that leverage elastic neural architectures and lightweight performance predictors to efficiently navigate large configuration spaces. Third, it develops a deployment-feedback-driven self-calibration framework that continuously detects deployment drift, refines adaptation behavior using runtime feedback, and sustains long-term multi-SLO satisfaction under evolving operating conditions. Collectively, these contributions transform edge AI inference from static resource orchestration into a self-improving adaptive system capable of reliable operation in dynamic edge environments.

References

- [1] Bart Cox, Jeroen Galjaard, Amirmasoud Ghiassi, Robert Birke, and Lydia Y. Chen. Masa: Responsive Multi-DNN Inference on the Edge. In *IEEE International Conference on Pervasive Computing and Communications (PerCom)*, Kassel, Germany, March, 2021.
- [2] Qianlin Liang, Walid A. Hanafy, Noman Bashir, Ahmed Ali-Eldin, David E. Irwin, and Prashant J. Shenoy. Dēlen: Enabling Flexible and Adaptive Model-serving for Multi-tenant Edge AI. In *ACM/IEEE Conference on Internet of Things Design and Implementation (IoTDI)*, San Antonio, TX, USA, May, 2023.
- [3] Achintya Kundu, Laura Wynter, Rhui Dih Lee, and Luis Angel D. Bathen. Transfer-Once-For-All: AI Model Optimization for Edge. In *IEEE International Conference on Edge Computing and Communications (EDGE)*, Chicago, IL, USA, July, 2023.
- [4] Dheeraj Ramchandani, Bahar Asgari, and Hyesoon Kim. Spica: Exploring FPGA Optimizations to Enable an Efficient SpMV Implementation for Computations at Edge. In *IEEE International Conference on Edge Computing and Communications (EDGE)*, Chicago, IL, USA, July, 2023.
- [5] Yifei Shen, Jiawei Shao, Xinjie Zhang, Zehong Lin, Hao Pan, Dongsheng Li, Jun Zhang, and Khaled B Letaief. Large language models empowered autonomous edge ai for connected intelligence. *IEEE Communications Magazine*, 62(10):140–146, 2024.
- [6] Dingzhu Wen, Peixi Liu, Guangxu Zhu, Yuanming Shi, Jie Xu, Yonina C Eldar, and Shuguang Cui. Task-oriented sensing, computation, and communication integration for multi-device edge ai. *IEEE Transactions on Wireless Communications*, 23(3):2486–2502, 2023.
- [7] Piyush Subedi, Jianwei Hao, In Kee Kim, and Lakshmish Ramaswamy. AI Multi-Tenancy on Edge: Concurrent Deep Learning Model Executions and Dynamic Model Placements on Edge Devices. In *IEEE International Conference on Cloud Computing (CLOUD)*, Sep., 2021.

- [8] Qianlin Liang, Prashant J. Shenoy, and David E. Irwin. AI on the Edge: Characterizing AI-based IoT Applications Using Specialized Edge Architectures. In *IEEE International Symposium on Workload Characterization (IISWC)*, Beijing, China, October, 2020.
- [9] Wonik Seo, Sanghoon Cha, Yeonjae Kim, Jaehyuk Huh, and Jongse Park. SLO-Aware Inference Scheduler for Heterogeneous Processors in Edge Platforms. *ACM Transactions on Architecture and Code Optimization (TACO)*, 18(4):1–26, 2021.
- [10] Ziyang Fu, Ju Ren, Deyu Zhang, Yuezhi Zhou, and Yaoxue Zhang. Kalmia: A Heterogeneous QoS-aware Scheduling Framework for DNN Tasks on Edge Servers. In *IEEE Conference on Computer Communications (INFOCOM)*, London, United Kingdom, May, 2022.
- [11] Liekang Zeng, Xu Chen, Zhi Zhou, Lei Yang, and Junshan Zhang. CoEdge: Cooperative DNN Inference With Adaptive Workload Partitioning Over Heterogeneous Edge Devices. *IEEE/ACM Transactions on Networking*, 29(2):595–608, 2020.
- [12] Neiwen Ling, Kai Wang, Yuze He, Guoliang Xing, and Daqi Xie. RT-mDL: Supporting Real-Time Mixed Deep Learning Tasks on Edge Platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, Coimbra, Portugal, November, 2021.
- [13] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. Band: Coordinated Multi-DNN Inference on Heterogeneous Mobile Processors. In *The Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, Portland, Oregon, USA, June, 2022.
- [14] Hyunseung Lee, Jihoon Hong, Soosung Kim, Seung Yul Lee, and Jae W. Lee. A Memory-Efficient Edge Inference Accelerator with XOR-based Model Compression. In *ACM/IEEE Design Automation Conference (DAC)*, San Francisco, CA, USA, July, 2023.
- [15] Lingdong Wang, Liyao Xiang, Jiayu Xu, Jiaju Chen, Xing Zhao, Dixi Yao, Xinbing Wang, and Baochun Li. Context-Aware Deep Model Compression for Edge Cloud Computing. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, Singapore, Nov., 2020.
- [16] Qianlin Liang, Walid A Hanafy, Ahmed Ali-Eldin, and Prashant Shenoy. Model-driven cluster resource management for ai workloads in edge clouds. *ACM Transactions on Autonomous and Adaptive Systems*, 18(1):1–26, 2023.
- [17] Ying He, Jingcheng Fang, F Richard Yu, and Victor C Leung. Large language models (llms) inference offloading and resource allocation in cloud-edge computing: An active inference approach. *IEEE Transactions on Mobile Computing*, 2024.
- [18] Puning Zhang, Fengyi Huang, Dapeng Wu, Boran Yang, Zhigang Yang, and Lei Tan. Device-edge-cloud collaborative acceleration method towards occluded face recognition in high-traffic areas. *IEEE Transactions on Multimedia*, 25:1513–1520, 2023.
- [19] Rakandhiya D. Rachmanto, Zaki Sukma, Ahmad N. L. Nabhaan, Arief Setyanto, Ting Jiang, and In Kee Kim. Characterizing Deep Learning Model Compression with Post-Training Quantization on Accelerated Edge Devices. In *IEEE International Conference on Edge Computing and Communications (EDGE)*, Shenzhen, China, July, 2024.
- [20] Yanfu Zhang, Shangqian Gao, and Heng Huang. Exploration and Estimation for Model Compression. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, Montreal, QC, Canada, October, 2021.

- [21] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. Branchynet: Fast inference via early exiting from deep neural networks. In *2016 23rd international conference on pattern recognition (ICPR)*, pages 2464–2469. IEEE, 2016.
- [22] Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E Gonzalez. Skipnet: Learning dynamic routing in convolutional networks. In *Proceedings of the European conference on computer vision (ECCV)*, pages 409–424, 2018.
- [23] Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. Dynamic neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(11):7436–7456, 2021.
- [24] Shaoshan Liu, Liangkai Liu, Jie Tang, Bo Yu, Yifan Wang, and Weisong Shi. Edge Computing for Autonomous Driving: Opportunities and Challenges. *Proceedings of IEEE*, 107(8):1697–1716, 2019.
- [25] Xiaojie Zhang, Houchao Gan, Amitangshu Pal, Soumyabrata Dey, and Saptarshi Debroy. On Balancing Latency and Quality of Edge-native Multi-view 3D Reconstruction. In *IEEE/ACM Symposium on Edge Computing (SEC)*, Wilmington, DE, USA, December, 2023.
- [26] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. EIE: Efficient Inference Engine on Compressed Deep Neural Network. In *ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, Seoul, South Korea, June, 2016.
- [27] Hyunseung Lee, Jihoon Hong, Soosung Kim, Seung Yul Lee, and Jae W Lee. A memory-efficient edge inference accelerator with xor-based model compression. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.
- [28] Xiaojie Zhang, Houchao Gan, Amitangshu Pal, Soumyabrata Dey, and Saptarshi Debroy. On balancing latency and quality of edge-native multi-view 3d reconstruction. In *2023 IEEE/ACM Symposium on Edge Computing (SEC)*, pages 1–13. IEEE, 2023.
- [29] Wang Lingdong, Xiang Liyao, Xu Jiayu, Chen Jiaju, Zhao Xing, Yao Dixi, Wang Xinbing, and Li Baochun. Context-aware deep model compression for edge cloud computing [c]. In *Proc of the 40th Int Conf on Distributed Computing Systems (ICDCS)*, pages 787–797. IEEE, 2020.
- [30] Yu Emma Wang, Gu-Yeon Wei, and David Brooks. Benchmarking tpu, gpu, and cpu platforms for deep learning. *arXiv preprint arXiv:1907.10701*, 2019.
- [31] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A Low-Latency Online Prediction Serving System. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, Boston, MA, USA, March, 2017.
- [32] NVIDIA Jetson Xavier. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-series/>.
- [33] Coral TPU USB Accelerator datasheet. <https://coral.ai/docs/accelerator/datasheet/>.
- [34] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going Deeper With Convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, June, 2015.

- [35] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *The International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019.
- [36] Mark Sandler, Andrew G. Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, June, 2018.
- [37] Forrest N. Iandola, Matthew W. Moskewicz, Khalid Ashraf, Song Han, William J. Dally, and Kurt Keutzer. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <1MB model size. *CoRR*, abs/1602.07360, 2016.
- [38] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the Inception Architecture for Computer Vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, June, 2016.
- [39] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *CoRR*, abs/1704.04861, 2017.
- [40] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding YOLO Series in 2021. *CoRR*, abs/2107.08430, 2021.
- [41] MoveNet: Ultra fast and accurate pose detection model. <https://www.tensorflow.org/hub/tutorials/movenet>.
- [42] George Papandreou, Tyler Zhu, Liang-Chieh Chen, Spyros Gidaris, Jonathan Tompson, and Kevin Murphy. PersonLab: Person Pose Estimation and Instance Segmentation with a Bottom-Up, Part-Based, Geometric Embedding Model. In *European Conference on Computer Vision (ECCV)*, Munich, Germany, September, 2018.
- [43] Sysstat. <http://sebastien.godard.pagesperso-orange.fr/>, 2021. [ONLINE].
- [44] jetson-stats 3.1.0. <https://pypi.org/project/jetson-stats/>, 2021. [ONLINE].
- [45] Li Lyna Zhang, Shihao Han, Jianyu Wei, Ningxin Zheng, Ting Cao, Yuqing Yang, and Yunxin Liu. Nn-meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, pages 81–93, 2021.
- [46] Qianlin Liang, Walid A Hanafy, Noman Bashir, Ahmed Ali-Eldin, David Irwin, and Prashant Shenoy. Dēlen: enabling flexible and adaptive model-serving for multi-tenant edge ai. In *Proceedings of the 8th ACM/IEEE Conference on Internet of Things Design and Implementation*, pages 209–221, 2023.
- [47] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [48] GeForce RTX 2080 Ti. <https://www.nvidia.com/en-us/geforce/20-series/>.
- [49] RTX A5000. <https://www.nvidia.com/en-us/products/workstations/rtx-a5000/>.

- [50] Lin Wang and Jingjing Zhang. Adaptive multi-armed bandit learning for task offloading in mobile edge computing. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5285–5289. IEEE, 2024.
- [51] Yijun Hao, Shusen Yang, Fang Li, Yifan Zhang, Shibo Wang, and Xuebin Ren. Edgetimer: adaptive multi-timescale scheduling in mobile edge computing with deep reinforcement learning. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, pages 671–680. IEEE, 2024.
- [52] Maximilian Wirth, Andrea Ortiz, and Anja Klein. Contextual multi-armed bandits for non-stationary heterogeneous mobile edge computing. In *GLOBECOM 2023-2023 IEEE Global Communications Conference*, pages 5599–5604. IEEE, 2023.
- [53] Romil Bhardwaj, Zhengxu Xia, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, Nikolaos Karianakis, Kevin Hsieh, Paramvir Bahl, and Ion Stoica. Ekyā: Continuous learning of video analytics models on edge compute servers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 119–135, 2022.
- [54] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 conference of the ACM special interest group on data communication*, pages 253–266, 2018.
- [55] Ting Jiang, Jianwei Hao, Sushruth Harsha, Rakandhiya D Rachmanto, Arief Setyanto, Lakshmish Ramaswamy, and In Kee Kim. Convergo: Multi-slo-aware scheduling for heterogeneous ai accelerators on edge devices. In *2025 IEEE International Conference on Edge Computing and Communications (EDGE)*, pages 115–125. IEEE, 2025.